

Guía de trabajo

Evaluación Sumativa N°2

Programación Orientada a Objeto Seguro

Unidad 2: Implementación de software orientado a objetos en Python

Docente	Michael Arjel
Sección / Código	114-2B-F1 / TI3V21
Ponderación	35% de la nota del módulo
Modalidad	Grupal (2 integrantes), mismo equipo y negocio de la Sumativa N°1
Plazo	Martes 6 de octubre de 2026, 23:59 (se evalúa el último commit antes del plazo)
Entrega	AAI, tarea 'Evaluación Sumativa N°2': informe PDF + enlace al repositorio GitHub
Puntaje	42 puntos (21 código + 21 informe). 25 puntos = nota 4.0
Alcance	Solo el modelo de clases del negocio en Python (sin base de datos ni interfaz)

1. Reglas críticas (pueden hacerte perder todo)

- **El repositorio debe ser público.** Si es privado o el enlace no abre, se considera *no entregado*.
- **El docente clona el repo y ejecuta main.py.** Lo que no funcione así se evalúa como no funcionando, aunque esté en el informe.
- Solo cuenta el **último commit antes del plazo** (martes 6/10, 23:59).
- Debes **documentar el uso de IA:** qué adoptaste, modificaste o descartaste, y por qué.
- El criterio 2.1.2 dice '**sin automatización del código**': deben poder entender y defender lo que entregan.
- La nota es la misma para todo el equipo. Una entrega por equipo.

2. Qué deben entregar

Actividad 1: Modelo en Python (carpeta model/)

- Una clase por archivo dentro de model/.
- **Herencia:** 3 subtipos que heredan de la clase base, llaman a `super().__init__()` y sobrescriben el método que se comporta distinto en cada uno.
- **Encapsulamiento:** atributos privados con `property` y validación en el setter del dato que la ficha exige validar.
- **Composición:** la parte se crea dentro del todo.
- **Agregación:** recibe un objeto que ya existe.
- La transacción contiene sus líneas de detalle.
- **Dos excepciones propias**, una por cada regla del negocio que impide una operación, con nombre del dominio y lanzadas desde el método que corresponde.
- Si descubren un error en el diagrama de la Sumativa N°1: corregirlo, actualizar la imagen en el repo y explicarlo en el informe.

Actividad 2: main.py en la raíz

Debe correr de principio a fin sin errores no controlados y:

- Crear objetos del negocio.
- Mostrar los **3 subtipos con su método distinto** (llamado de forma polimórfica).
- Mostrar el **dato con validación** (incluye un intento inválido capturado).
- Mostrar la **transacción con sus líneas de detalle**.
- Provocar **a propósito las 2 reglas del negocio**, capturarlas con `try/except` específico (no `except Exception` genérico), informar el error y seguir ejecutándose.

Actividad 3: Informe PDF (máx. 8 páginas sin contar la portada)

Cada afirmación con evidencia (captura de main.py o fragmento breve de código indicando el archivo). Orden obligatorio:

1. **Portada:** nombre del equipo, integrantes, negocio y enlace al repositorio.
2. **Diagrama y código:** diagrama de clases (versión nueva y motivo si cambió) + fragmento de código de herencia, agregación y composición, con explicación breve de por qué se programó así.
3. **Evidencia de funcionamiento:** capturas de main.py de los 3 subtipos, el dato con validación, la transacción con líneas de detalle y las 2 reglas capturadas, indicando clase y método de cada una.
4. **Uso de inteligencia artificial:** qué partes propuso la IA + al menos un ejemplo concreto de algo adoptado, modificado o descartado, con la razón técnica.

Estructura sugerida del repositorio

```
mi-repo/
|-- main.py
|-- diagrama_clases.png
|-- README.md
|-- model/
    |-- __init__.py
    |-- (una clase por archivo)
    |-- (excepciones propias, una por regla)
```

3. Cómo se evalúa (para el máximo de 7 pts por criterio)

Criterio	Qué se necesita para el nivel Destacado
Código 2.1.1	Todas las clases del diagrama con atributos y métodos; cada relación según su tipo; main.py demuestra los 4 requisitos (subtipos, validación, transacción con detalle, reglas); todo cambio explicado.
Código 2.1.2	Subtipos con super().__init__() y método sobrescrito; atributos privados + property + validación en el setter; el código que usa los objetos llama al método polimórfico sin preguntar el tipo (nada de if/instance).
Código 2.1.4	Cada regla con excepción propia, nombre del dominio, lanzada desde el método correcto; main.py la captura con try/except y sigue ejecutándose.
Informe 2.1.1	Diagrama + fragmento de código de cada relación (herencia, agregación, composición) con explicación; cambios respecto de la Sumativa N°1 justificados.
Informe 2.1.2	Capturas de main.py con los 3 subtipos, dato con validación, transacción con líneas de detalle y 2 reglas capturadas, indicando clase y método.
Informe 2.1.5	Indica qué partes propuso la IA y da un ejemplo concreto de algo adoptado, modificado o descartado, con la razón técnica.

Niveles: Destacado 7 pts, Habilitado 5, En desarrollo 3, No logrado 1. Total 6 criterios = 42 puntos. Errores típicos que bajan puntaje: un subtipo sin super(), validación fuera del setter, if/instance en vez de polimorfismo, except genérico, print/return False en vez de excepciones.

Tabla de puntaje y nota (60% de exigencia)

Pts	Nota	Pts	Nota	Pts	Nota	Pts	Nota
0	1.0	11	2.3	22	3.6	33	5.4
1	1.1	12	2.4	23	3.8	34	5.6
2	1.2	13	2.6	24	3.9	35	5.8
3	1.4	14	2.7	25	4.0	36	5.9
4	1.5	15	2.8	26	4.2	37	6.1
5	1.6	16	2.9	27	4.4	38	6.3
6	1.7	17	3.0	28	4.5	39	6.5
7	1.8	18	3.2	29	4.7	40	6.6
8	2.0	19	3.3	30	4.9	41	6.8
9	2.1	20	3.4	31	5.1	42	7.0
10	2.2	21	3.5	32	5.2		

Nota 4.0 = 25 puntos (destacado).

4. Qué necesito reunir antes de pedirle código a la IA

El enunciado no incluye su negocio ni su diagrama. Junten estos datos (salen del diagrama de la Sumativa N°1):

- [] Diagrama de clases de la Sumativa N°1 (imagen o descripción).
- [] Nombre del **negocio**, la **clase base** y los **3 subtipos**.
- [] El **método que varía** entre subtipos.
- [] El **dato que se valida** y la regla de validación.
- [] Las **2 reglas del negocio** que impiden una operación.
- [] Qué relaciones son **composición**, cuáles **agregación**, y cuál es la **transacción** con sus líneas de detalle.

5. Prompts listos para usar con la IA

Reemplacen lo que está entre [corchetes] con los datos de su negocio. Úsenlos en orden y pídanle a la IA que explique el código para que puedan defenderlo.

Prompt 1: Contexto general (pegar primero)

Soy estudiante de Ingeniería Informática (INACAP) y debo implementar en Python el modelo de clases de mi negocio "[NOMBRE DEL NEGOCIO]". Solo se evalúa el modelo de clases, sin base de datos ni interfaz gráfica.

Reglas obligatorias:

- Una clase por archivo dentro de la carpeta model/.
- Atributos privados (con guion bajo) expuestos mediante @property, con validación en el setter para: [DATO A VALIDAR y su regla].
- Herencia: la clase base [CLASE BASE] y tres subtipos [SUBTIPO 1, 2, 3]. Cada subtipo debe llamar a super().__init__() y sobrescribir el método [MÉTODO QUE VARÍA], que se comporta distinto en cada uno.
- Polimorfismo real: el código que usa los objetos debe llamar al método sin preguntar el tipo (prohibido usar if/instance para decidir).
- Composición: [TODO] crea sus [PARTES] dentro de su propio constructor.
- Agregación: [CLASE] recibe por parámetro un objeto [CLASE] que ya existe.
- La transacción [TRANSACCIÓN] contiene una lista de líneas de detalle [LÍNEA DETALLE].
- Dos excepciones propias con nombre del dominio (no genéricas), heredando de Exception, una por cada regla que impide una operación:
Regla 1: [DESCRIPCIÓN]. Regla 2: [DESCRIPCIÓN].
Deben lanzarse (raise) desde el método que corresponde.
- Código en español, con nombres claros, sin librerías externas.

Antes de escribir código, confirma que entendiste y lista las clases, atributos, métodos y relaciones que vas a crear.

Prompt 2: Generar las clases

Ahora genera el código de cada clase de la carpeta model/, un archivo por clase, indicando el nombre del archivo antes de cada bloque de código. Incluye las excepciones propias. Después de cada clase, explica en 2 líneas qué principio de POO aplica (herencia, encapsulamiento, polimorfismo, composición o agregación), porque debo justificarlo en un informe.

Prompt 3: Generar main.py

Escribe main.py para la raíz del repositorio. Debe ejecutarse de principio a fin sin errores no controlados e imprimir con títulos claros:

1. Los tres subtipos, llamando en un ciclo al método [MÉTODO QUE VARÍA] de forma polimórfica (sin isinstance).
2. El dato con validación: un caso válido y uno inválido capturado con try/except (ValueError u otra excepción específica).

3. La transacción con sus líneas de detalle.
 4. Las dos reglas del negocio provocadas a propósito y capturadas cada una con su excepción propia (nunca `except Exception` genérico), mostrando el mensaje de error y continuando la ejecución.
- Importa desde `model/` correctamente para que funcione al clonar el repo.

Prompt 4: Revisión contra la rúbrica (muy importante)

Revisa todo el código anterior contra esta lista y dime qué falla:

- ¿Cada subtipo llama a `super().__init__()`?
 - ¿Todos los atributos son privados con `@property`, y la validación está dentro del setter (no fuera)?
 - ¿El polimorfismo se usa sin `if/ isinstance`?
 - ¿La composición crea la parte dentro del todo y la agregación recibe el objeto por parámetro?
 - ¿Cada regla tiene su excepción propia lanzada desde el método correcto?
 - ¿`main.py` captura con `except` específico y sigue ejecutándose?
 - ¿Cada clase está en su propio archivo en `model/`?
 - ¿Hay `imports` que fallarían al clonar el repo y ejecutar `python main.py`?
- Corrige lo que encuentres y explica el cambio.

Prompt 5: Documentar el uso de IA (para el informe)

Necesito documentar el uso de IA. Con base en esta conversación, dame una tabla con: qué parte del código propusiste, qué decisión tomamos (adoptado, modificado o descartado) y la razón técnica. Destaca al menos un ejemplo donde una propuesta tuya se corrigió o descartó y por qué.

Consejo: la rúbrica premia un ejemplo concreto con razón técnica, no una frase general. Anoten durante el proceso lo que la IA propuso mal (un `except Exception` genérico, un setter sin validación, un `isinstance`, un subtipo sin `super()`) y cómo lo corrigieron.

6. Checklist final antes de entregar

- [] Repositorio **público**; probé abrir el enlace en una ventana de incógnito.
- [] Cloné el repo en una carpeta nueva y `python main.py` corre completo sin errores.
- [] Una clase por archivo en `model/` e imagen del diagrama en el repo.
- [] Último commit hecho **antes del martes 6/10 a las 23:59**.
- [] Informe PDF de máximo 8 páginas (sin portada), en el orden pedido, con capturas que indican clase y método.
- [] Sección de IA con ejemplo concreto y razón técnica.
- [] Subí el PDF y el enlace en el AAI (una entrega por equipo).

Siguiente paso: entregar a Claude el diagrama de clases de la Sumativa N°1 y los datos del negocio para generar el código (`model/`, excepciones y `main.py`) y la estructura del informe.